

REQUISITOS DE DESARROLLO · V1.0 · JUNIO 2026

Elfie

PRTS Desktop — Lista de Requisitos

Todo lo que necesitás instalar, descargar, configurar y considerar para desarrollar Elfie Desktop. Organizado por categoría, fase de uso y prioridad.

TAURI 2.X

RUST + CARGO

OLLAMA

WHISPER.CPP

KOKORO TTS

PORCUPINE

LANCEDB

CUDA 12.X

WINDOWS 10/11

PROPIETARIO: SYLFT · IA: ELFIE · RTX 5060 · 8 GB VRAM · 16 GB RAM · RYZEN 5 5600G

Resumen Ejecutivo

CÓMO LEER ESTE DOCUMENTO

Los requisitos están marcados por prioridad: **OBLIGATORIO** — sin esto no compila o no funciona ·

RECOMENDADO — necesario para las funciones principales · **OPCIONAL** — mejoras o fases avanzadas ·

CONDICIONAL — solo si se usa esa feature específica.

Conteo de requisitos

CATEGORÍA	ITEMS	DESCARGA	ESPACIO APROX.	COSTO
Herramientas de desarrollo (Rust, Node, MSVC)	4	Sí	~10 GB	Gratuito
Framework Tauri + plugins + crates Rust	12	Cargo	~2 GB cache	Gratuito
IA Local — Ollama + modelos LLM	5	Sí	~13 GB	Gratuito
STT — Whisper.cpp + modelo	2	Sí	~1.6 GB	Gratuito
TTS — motores de voz	4	Sí/pip	~1 GB	Gratuito
Wake word — Picovoice Porcupine	3	pip + cuenta	<50 MB	Gratuito
GPU — CUDA + drivers	3	Sí	~4 GB	Gratuito
Python + entorno de sidecars	8 paquetes	pip	~3 GB	Gratuito
RAG local — LanceDB (Fase 6)	2	pip	~500 MB	Gratuito
Cuentas y API Keys	4	—	—	Mayormente gratuito
TOTAL MÍNIMO (Fases 0–5)	—	—	~25 GB libres	\$0

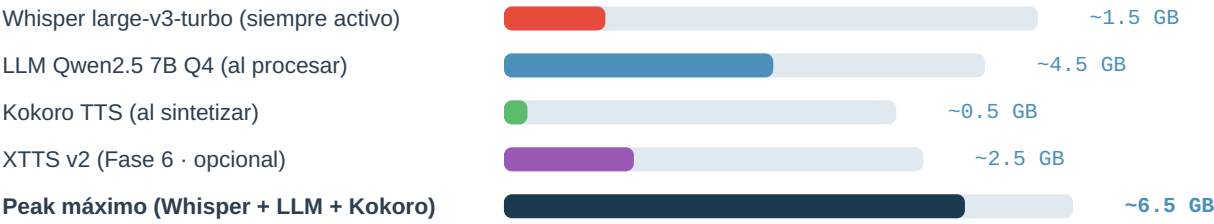
Espacio en disco — Desglose

Modelos Ollama (4 modelos LLM)		~13 GB
CUDA Toolkit + cuDNN		~3.5 GB
Build Tools MSVC (Rust en Windows)		~7 GB
Whisper large-v3-turbo (modelo STT)		~1.6 GB
TTS models (Kokoro + Piper + XTTS v2)		~3 GB
Cargo cache (crates Rust)		~2 GB
Python + paquetes pip		~1.5 GB

RECOMENDACIÓN DE DISCO

Tener al menos **30 GB libres** antes de empezar. Los modelos de Ollama por defecto se guardan en `C:\Users\Sylft\.ollama\` — si el disco C tiene poco espacio, cambiar la ruta con la variable de entorno `OLLAMA_MODELS=D:\elfie\models` antes de instalar Ollama.

Presupuesto de VRAM (RTX 5060 · 8 GB)



Herramientas de Desarrollo Base

Rust — Lenguaje del backend de Tauri

INSTALACIÓN

URL: <https://rustup.rs> — instalador único para Windows

Instala automáticamente: rustc · cargo · rustfmt · clippy · rust-analyzer

Versión mínima: stable 1.77+ (rustup instala la última stable por defecto)

Peso: ~800 MB (toolchain) + ~2 GB cache de crates al compilar Tauri por primera vez

```
# Verificar instalación (en PowerShell, tras reiniciar) rustc --version # rustc 1.xx.0 (xxxxxxx  
YYYY-MM-DD) cargo --version # cargo 1.xx.0 (xxxxxxx YYYY-MM-DD)
```

PRIMER BUILD DE TAURI

La primera compilación descarga y compila todas las crates dependencias de Tauri. Tarda **10–20 minutos** dependiendo del procesador. Las compilaciones siguientes son incrementales y rápidas (~30 s).

Microsoft C++ Build Tools — Obligatorio para compilar Rust en Windows

DATO	VALOR
URL	https://visualstudio.microsoft.com/visual-cpp-build-tools/
Componente a seleccionar	"Desarrollo de escritorio con C++" (workload completo)
Incluye	MSVC · Windows SDK · CMake · MSBuild · libclang (para whisper-rs)
Peso	~7 GB
Prioridad	OBLIGATORIO — sin esto Rust no compila en Windows

Node.js

DATO	VALOR
URL	https://nodejs.org — versión LTS (22.x recomendada)
Uso en el proyecto	Tauri CLI · scripts de package.json · npm
Peso	~80 MB
Prioridad	OBLIGATORIO

Tauri CLI

```
# Instalar Tauri CLI (elegir una de las dos formas) npm install -g @tauri-apps/cli@^2 # vía npm
(recomendado) cargo install tauri-cli # vía Cargo (alternativa) # Inicializar el proyecto dentro
del repo existente cd mi-app-PRTS mkdir elfie-desktop && cd elfie-desktop npx @tauri-apps/cli
init # sigue el wizard
```

LLVM / Clang — Para compilar whisper-rs

DATO	VALOR
Instalación rápida	<code>winget install LLVM.LLVM</code> (desde PowerShell)
Alternativa	Incluido en Visual Studio Build Tools si se marca "Clang"
Necesario para	La crate <code>whisper-rs</code> requiere libclang en PATH
Verificar	<code>clang --version</code>
Prioridad	RECOMENDADO (obligatorio en Fase 3)

Extensiones de VS Code recomendadas

EXTENSIÓN	ID	FUNCIÓN
rust-analyzer	rust-lang.rust-analyzer	Autocompletado, errores inline en Rust
Tauri	tauri-apps.tauri-vscode	Comandos Tauri integrados en VS Code
Even Better TOML	tamasfe.even-better-toml	Syntax de Cargo.toml
CodeLLDB	vadimcn.vscode-lldb	Debugger para Rust

Tauri 2.x — Plugins y Crates Rust

SIN DESCARGA MANUAL

Todos estos se declaran en `cargo.toml` y se descargan automáticamente con `cargo build`. No hay instaladores separados.

Plugins Tauri 2.x

PLUGIN	FUNCIÓN	FASE	PRIORIDAD
<code>tauri-plugin-global-shortcut</code>	Atajos de teclado globales (Ctrl+Space, Alt+E)	F1	OBLIGATORIO
<code>tauri-plugin-autostart</code>	Auto-inicio con Windows al encender	F1	RECOMENDADO
<code>tauri-plugin-notification</code>	Notificaciones nativas de Windows	F1	RECOMENDADO
<code>tauri-plugin-clipboard-manager</code>	Leer y escribir el portapapeles	F2	RECOMENDADO
<code>tauri-plugin-fs</code>	Acceso al sistema de archivos (abrir carpetas)	F2	RECOMENDADO
<code>tauri-plugin-shell</code>	Ejecutar procesos externos (sidecars Python)	F3	RECOMENDADO
<code>tauri-plugin-sql</code>	SQLite local (si se prefiere sobre LanceDB)	F6	OPCIONAL
<code>tauri-plugin-updater</code>	Auto-actualizaciones (no prioritario)	—	DESCARTADO

Crates Rust (dependencias en Cargo.toml)

CRATE	FUNCIÓN	FASE	PRIORIDAD
<code>sysinfo</code>	Métricas de CPU y RAM en tiempo real	F2	RECOMENDADO
<code>nvml-wrapper</code>	Métricas GPU NVIDIA (RTX 5060): VRAM, temp, uso	F2	RECOMENDADO
<code>cpal</code>	Captura de audio del micrófono	F3	RECOMENDADO
<code>whisper-rs</code>	Bindings de Whisper.cpp para STT local	F3	RECOMENDADO
<code>request</code>	Cliente HTTP para la API de Ollama (localhost)	F4	RECOMENDADO
<code>tokio</code>	Runtime async para operaciones concurrentes	F0	OBLIGATORIO
<code>serde / serde_json</code>	Serialización JSON (IPC frontend ↔ backend)	F0	OBLIGATORIO
<code>windows</code>	APIs de Windows para control de volumen y procesos	F2	RECOMENDADO

Configuración mínima de Cargo.toml

```
[package] name = "elfie-desktop" version = "0.1.0" edition = "2021" [dependencies] tauri = { version = "2", features = ["tray-icon", "image-png"] } tokio = { version = "1", features =
```

```
["full"] } serde = { version = "1", features = ["derive"] } serde_json = "1" reqwest = { version  
= "0.11", features = ["json"] } sysinfo = "0.30" nvml-wrapper = "0.10" # F2: métricas RTX 5060  
cpal = "0.15" # F3: captura de micrófono whisper-rs = "0.11" # F3: STT local  
[target.'cfg(windows)'.dependencies] windows = { version = "0.52", features =  
["Win32_UI_WindowsAndMessaging", "Win32_Media_Audio", "Win32_System_ProcessStatus"] }
```

Ollama — Runtime de LLMs

DATO	VALOR
URL de descarga	https://ollama.ai/download (instalador Windows)
Peso del instalador	~100 MB
Qué instala	Servidor Ollama · CLI · runtime de modelos · API en localhost:11434
Requisito de GPU	CUDA 12.x instalado previamente (sección 07)
Directorio de modelos	<code>C:\Users\Sylft\.ollama\models\</code> (configurable)
Cambiar directorio	Variable de entorno: <code>OLLAMA_MODELS=D:\elfie\models</code>
Prioridad	RECOMENDADO (necesario en Fase 4)

Modelos LLM a descargar

MODELO	COMANDO	DISCO	VRAM	VELOCIDAD	USO PRINCIPAL	FASE
Phi-3.5 Mini Q4	<code>ollama pull phi3.5</code>	~2.2 GB	~2.5 GB	~120 tok/s	Router · comandos rápidos	F4
Qwen2.5 7B Q4	<code>ollama pull qwen2.5:7b</code>	~4.7 GB	~4.5 GB	~50 tok/s	Inbox · clasificación · es-MX	F4
Llama 3.1 8B Q4	<code>ollama pull llama3.1:8b</code>	~4.9 GB	~5.0 GB	~40 tok/s	Conversación profunda · ask	F4
nomic-embed-text	<code>ollama pull nomic-embed-text</code>	~274 MB	~270 MB	—	Embeddings para RAG (Apuntes)	F6

ESTRATEGIA DE DESCARGA RECOMENDADA

Fases 0–3: No descargar nada aún. Ollama solo se necesita en Fase 4.

Inicio Fase 4: Descargar primero `phi3.5` (más pequeño, para validar que Ollama funciona con la RTX 5060).

Después: Descargar `qwen2.5:7b` para inbox y clasificación.

Solo si se necesita: `llama3.1:8b` para preguntas complejas.

Fase 6 únicamente: `nomic-embed-text` para los embeddings del RAG.

```
# Verificar que Ollama usa la GPU (tras instalación) ollama run phi3.5 "hola" # En la respuesta debe aparecer: "using GPU" o similar # Si dice "using CPU" → revisar drivers CUDA (sección 07) # Ver modelos instalados ollama list # API REST (para Tauri) curl http://localhost:11434/api/generate -d '{"model": "phi3.5", "prompt": "hola", "stream": false}'
```


STT — Whisper.cpp (Reconocimiento de Voz Local)

QUÉ REEMPLAZA

Whisper.cpp reemplaza `webkitSpeechRecognition` que actualmente envía el audio a los servidores de Google. Con Whisper, el audio nunca sale de la computadora, funciona sin internet y tiene mayor precisión en español mexicano.

Opción A — whisper-rs (Rust nativo, recomendado)

DATO	VALOR
Crate	<code>whisper-rs = "0.11"</code> en Cargo.toml
Prerequisito	LLVM/Clang en PATH (<code>winget install LLVM.LLVM</code>)
Prerequisito	CUDA Toolkit instalado para aceleración GPU
Ventaja	Integración directa en Rust, sin proceso Python separado
Desventaja	Compilación más compleja en Windows
Prioridad	RECOMENDADO para Fase 3

Opción B — faster-whisper (Python sidecar, más simple para empezar)

```
pip install faster-whisper # implementación optimizada con CTranslate2 pip install pyaudio #
captura de micrófono en Python
```

Modelo de Whisper a descargar

MODELO	CALIDAD	VRAM	DISCO	LATENCIA (FRASE ~5 S)	RECOMENDACIÓN
large-v3-turbo	Excelente	~1.5 GB	~1.6 GB	~300 ms (GPU)	RECOMENDADO
medium	Muy buena	~1.5 GB	~1.5 GB	~400 ms	Alternativa si large-v3-turbo falla
small	Buena	~0.5 GB	~500 MB	~150 ms	Solo si la VRAM escasea mucho

DESCARGA DEL MODELO (PARA WHISPER-RS)

Los modelos en formato ggml se descargan de Hugging Face:

<https://huggingface.co/ggerganov/whisper.cpp/resolve/main/ggml-large-v3-turbo.bin>

Guardar en: `elfie-desktop/models/whisper/ggml-large-v3-turbo.bin`

Con faster-whisper (Python), el modelo se descarga automáticamente al primer uso.

TTS — Motores de Voz de Elfie

Kokoro TTS — Motor primario (local, GPU)

DATO	VALOR
Instalación	<code>pip install kokoro-onnx onnxruntime-gpu</code>
Modelos	Se descargan automáticamente al primer uso (~300 MB)
VRAM al ejecutar	~0.5 GB
Latencia	<100 ms en GPU
Voz es-MX	Disponible
Clonación de voz	No
Prioridad	RECOMENDADO — motor principal

Piper TTS — Fallback CPU

DATO	VALOR
Instalación	Binario precompilado para Windows
URL	https://github.com/rhasspy/piper/releases — <code>piper_windows_amd64.zip</code>
Modelos de voz es-MX	Descargar <code>.onnx</code> + <code>.json</code> del repositorio de modelos de Piper
VRAM al ejecutar	0 — corre solo en CPU
Latencia	<50 ms en CPU
Uso	Cuando la GPU está ocupada con el LLM
Prioridad	RECOMENDADO — fallback

edge-tts — Fallback Cloud Gratuito

DATO	VALOR
Instalación	<code>pip install edge-tts</code>
Uso	Sin modelos locales — llama a Microsoft en la nube
API key	No requiere
Voz es-MX recomendada	<code>es-MX-DaliaNeural</code> (misma wake word)
Costo	Gratuito (500k caracteres/mes con cuenta Microsoft, ilimitado sin cuenta)
Latencia	~200 ms (requiere internet)

DATO	VALOR
Prioridad	RECOMENDADO — fallback cloud

XTTS v2 (Coqui) — Clonación de voz (Fase 6)

DATO	VALOR
Instalación	<code>pip install TTS</code>
Modelos	Se descargan al primer uso (~2 GB)
VRAM al ejecutar	~2.5 GB
Capacidad especial	Clonar cualquier voz con 5 segundos de audio de referencia
Uso en Elfie	Personalizar la voz de Elfie grabando una voz de referencia
Prioridad	OPCIONAL — Fase 6

ESTRATEGIA EN CAPAS – ORDEN DE PRIORIDAD

- 1 • **Kokoro** (local GPU) → respuestas cotidianas, <100 ms, \$0
- 2 • **Piper** (local CPU) → si GPU saturada con LLM, <50 ms, \$0
- 3 • **edge-tts DaliaNeural** → si modelos no están cargados, ~200 ms, \$0
- 4 • **XTTS v2** (Fase 6) → voz clonada para identidad de Elfie

Wake Word — Picovoice Porcupine

QUÉ REEMPLAZA

Reemplaza `webkitSpeechRecognition` continuo en el navegador. Porcupine detecta la palabra clave directamente en el dispositivo (on-device), sin enviar audio a ningún servidor. Funciona con PRTS cerrado, consume menos del 3% de CPU y es el único componente que requiere una cuenta externa.

Cuenta Picovoice (AccessKey)

DATO	VALOR
URL de registro	<code>https://console.picovoice.ai</code>
Plan necesario	Free (gratuito, suficiente para uso personal)
Qué incluye el plan Free	AccessKey · hasta 3 palabras clave custom · uso personal
Lo que se obtiene	AccessKey (string) que va en el código — no es una API key cloud, autoriza el SDK local
Prioridad	RECOMENDADO (necesario en Fase 3)

Modelo de palabra clave "Dalia"

PASO CRÍTICO — ENTRENAR EL MODELO CUSTOM

"Dalia" no existe en los presets estándar de Porcupine (que son en inglés). Hay que entrenar un modelo custom:

1. Ir a **console.picovoice.ai** → **Porcupine** → **Train Keyword**
2. Escribir la palabra: **"Dalia"** (o "hey Dalia" para mayor precisión)
3. Seleccionar idioma: **Español**
4. Descargar el archivo `.ppn` generado
5. Guardar en `elfie-desktop/models/porcupine/dalia_es.ppn`

SDK de Porcupine

MÉTODO	INSTALACIÓN	VENTAJA	RECOMENDADO
Python sidecar	<code>pip install pvporcupine</code>	Más simple, documentación amplia	sí para empezar
Rust nativo	<code>pv-porcupine</code> en Cargo.toml	Integración directa, sin Python	Fases avanzadas

```
# Python sidecar - ejemplo de uso import pvporcupine, pyaudio, struct
porcupine = pvporcupine.create( access_key="TU_ACCESS_KEY_AQUI", keyword_paths=["models/porcupine/
dalia_es.ppn"] ) # al detectar "Dalia" - emite evento al frontend de Tauri
```

GPU — CUDA, cuDNN y Drivers NVIDIA

VERIFICAR PRIMERO — RTX 5060 EN WINDOWS 10

La RTX 5060 (arquitectura Blackwell, 2025) es posterior al fin de soporte de Windows 10 (octubre 2025). Antes de instalar CUDA, verificar que NVIDIA ofrece drivers para RTX 5060 en Windows 10. Si no los hay, la solución es actualizar a Windows 11 antes de Fase 3.

Drivers NVIDIA

DATO	VALOR
URL	https://www.nvidia.com/drivers
Seleccionar	GeForce RTX 5060 · Windows 10/11 64-bit · Game Ready Driver
Verificar en PowerShell	<code>nvidia-smi</code> — debe mostrar la RTX 5060 con su VRAM
Prioridad	OBLIGATORIO — prerequisite de todo lo demás

CUDA Toolkit

DATO	VALOR
URL	https://developer.nvidia.com/cuda-downloads
Versión	CUDA 12.x (la más reciente compatible con tu driver)
Tipo de instalador	Local (exe) — más confiable en Windows
Peso	~3–4 GB
Verificar	<code>nvcc --version</code> en PowerShell
Necesario para	Ollama GPU · faster-whisper · kokoro-onnx GPU · XTTS v2
Prioridad	RECOMENDADO (sin CUDA todo corre en CPU, mucho más lento)

cuDNN

DATO	VALOR
URL	https://developer.nvidia.com/cudnn
Requiere	Cuenta NVIDIA Developer (gratuita)
Instalación	Copiar archivos DLL/lib/include a la carpeta de CUDA — no tiene instalador
Versión	cuDNN 9.x compatible con CUDA 12.x
Necesario para	Kokoro TTS · XTTS v2 (los modelos TTS se benefician de cuDNN)

DATO

VALOR

Prioridad **OPCIONAL** — mejora rendimiento de TTS, no es bloqueante

```
# Verificación completa del entorno GPU (PowerShell) nvidia-smi # driver + VRAM disponible nvcc
--version # CUDA instalado python -c "import torch; print(torch.cuda.is_available())" # True si
PyTorch detecta CUDA ollama run phi3.5 "test" # verificar que Ollama usa GPU
```


Python — Entorno de Sidecars

ROL DE PYTHON EN ELFIE DESKTOP

Python no es el lenguaje principal — ese es Rust. Python actúa como "sidecar": procesos separados que Tauri lanza y controla para los componentes de IA que aún no tienen bindings estables en Rust (Porcupine, Kokoro TTS, faster-whisper). A largo plazo se pueden migrar a Rust puro.

Python base

DATO	VALOR
URL	https://python.org/downloads
Versión recomendada	Python 3.11 (mayor compatibilidad con TTS · Coqui aún no soporta 3.12 completamente)
Opciones al instalar	Marcar "Add Python to PATH" — sin esto nada funciona desde PowerShell
Verificar	<code>python --version</code> · <code>pip --version</code>
Prioridad	RECOMENDADO

Paquetes pip — Instalación completa por fases

```
# FASE 3 — STT + Wake Word pip install faster-whisper pyaudio pvporcupine # FASE 3 — TTS primario y fallbacks pip install kokoro-onnx onnxruntime-gpu edge-tts # FASE 3 — Piper (binario separado, ver sección 06) # Solo pip install piper-tts si se quiere desde Python pip install piper-tts # FASE 6 — RAG Local pip install lancedb sentence-transformers # FASE 6 — Clonación de voz (XTTS v2) pip install TTS # paquete de Coqui AI (~2 GB modelos al primer uso) # UTILIDADES GENERALES pip install numpy scipy soundfile
```

Tabla completa de paquetes pip

PAQUETE	FUNCIÓN	FASE	PRIORIDAD
faster-whisper	STT — transcripción de voz local	F3	RECOMENDADO
pyaudio	Captura de micrófono en Python	F3	RECOMENDADO
pvporcupine	Wake word Porcupine SDK	F3	RECOMENDADO
kokoro-onnx	TTS primario Kokoro	F3	RECOMENDADO
onnxruntime-gpu	Runtime ONNX con aceleración GPU	F3	RECOMENDADO
edge-tts	TTS cloud gratuito (DaliaNeural)	F3	RECOMENDADO
numpy · scipy · soundfile	Utilidades de audio	F3	RECOMENDADO
lancedb	Vector store para RAG local	F6	OPCIONAL

PAQUETE	FUNCIÓN	FASE	PRIORIDAD
sentence-transformers	Embeddings alternativos a Ollama	F6	OPCIONAL
TTS (Coqui)	XTTS v2 — clonación de voz	F6	OPCIONAL

ENTORNO VIRTUAL RECOMENDADO

Crear un `venv` dedicado para los sidecars de Elfie para evitar conflictos con otros proyectos Python:

```
python -m venv elfie-desktop\venv .\elfie-desktop\venv\Scripts\activate pip install faster-whisper pyaudio pvporcupine kokoro-onnx onnxruntime-gpu edge-tts numpy scipy soundfile
```

RAG Local — LanceDB (Fase 6)

ALCANCE DEFINIDO

RAG solo se usa para el módulo **Apuntes**. El resto de los módulos (tareas, gym, finanzas, dieta) son datos estructurados que el LLM consulta directamente vía SQL. RAG no aporta nada sobre ellos.

LanceDB

DATO	VALOR
Instalación Python	<code>pip install lancedb</code>
Crate Rust	<code>lancedb = "0.9"</code> (alternativa nativa)
Tipo	Vector store embebido — como SQLite pero para vectores semánticos
Almacenamiento	En disco local: <code>elfie-desktop/db/apuntes.lance/</code>
Sin servidor	No requiere Docker, proceso separado ni configuración
Privacidad	Las notas nunca salen de la computadora
Prioridad	OPCIONAL — solo Fase 6

Motor de embeddings

OPCIÓN	INSTALACIÓN	VENTAJA	VRAM
nomic-embed-text via Ollama	<code>ollama pull nomic-embed-text</code>	Ya en el stack de Ollama, sin paquete extra	~270 MB
sentence-transformers	<code>pip install sentence-transformers</code>	Más opciones de modelos multilingüe	~500 MB

Supabase — Migraciones Nuevas

CLI de Supabase

```
# Verificar que está instalado supabase --version # Si no está instalado (Windows, con scoop)
scoop bucket add supabase https://github.com/supabase/scoop-bucket.git scoop install supabase
```

Nuevas migraciones a crear y aplicar

```
# Crear las migraciones (en orden) supabase migration new elfie_config # tabla configuración de
personalidad y voz supabase migration new voice_profiles # perfiles de voz clonados supabase
migration new system_actions # log de acciones del SO supabase migration new system_metrics #
historial CPU/GPU/RAM supabase migration new routines # Routine Engine Visual # Aplicar a la BD
de producción supabase db push
```

MIGRACIÓN	TABLA	FASE	DESCRIPCIÓN
0012_elfie_config	elfie_config	F4	Personalidad, motor de voz, modelo LLM local
0013_voice_profiles	voice_profiles	F6	Perfiles de voz clonados (XTTS v2)
0014_system_actions	system_actions	F2	Log de acciones del SO (abrir apps, volumen...)
0015_system_metrics	system_metrics	F2	Historial de métricas para gráficas
0016_routines	routines	F5	Rutinas configurables del Routine Engine

Cuentas y API Keys

SERVICIO	PARA QUÉ	ESTADO	COSTO	ACCIÓN REQUERIDA
Anthropic	Claude API (briefing, insights)	YA CONFIGURADO	Ya pagado	Ninguna
Supabase	Base de datos compartida	YA CONFIGURADO	Plan gratuito	Ninguna
Google Cloud	Google Calendar (recordatorios)	YA CONFIGURADO	Gratuito	Agregar localhost + URL Tauri a orígenes autorizados
Picovoice Console	AccessKey para Porcupine wake word	PENDIENTE	Gratuito	Registrarse en console.picovoice.ai · entrenar modelo "Dalia"
NVIDIA Developer	Descargar cuDNN	PENDIENTE	Gratuito	Registrarse en developer.nvidia.com
ElevenLabs	TTS premium (voz clonada cloud)	OPCIONAL	\$5–22/mes	Solo si se quiere TTS cloud de alta calidad
Hugging Face	Descargar modelo Whisper ggml	SIN CUENTA	Gratuito	Descarga directa sin registro

GOOGLE CLOUD – ACTUALIZAR ORÍGENES AUTORIZADOS

El ID de cliente OAuth actual (`config.js`) tiene autorizados `https://tu-app.vercel.app` y `http://localhost:3210` . Tauri expone el WebView en un origen diferente (`tauri://localhost`). Agregar `tauri://localhost` a los orígenes JS autorizados en Google Cloud Console para que el módulo de Recordatorios siga funcionando en el desktop.

Consideraciones Extras y Posibles Problemas

Windows Defender / Antivirus

FALSOS POSITIVOS ESPERADOS

Ollama, los ejecutables de Tauri compilados localmente y los binarios de Piper/Porcupine pueden ser marcados como amenaza por Windows Defender al primer uso. Solución: agregar las carpetas del proyecto a las exclusiones de Defender antes de compilar.

```
# PowerShell (como Administrador) — agregar exclusiones a Defender
Add-MpPreference -ExclusionPath "C:\Users\Sylft\Desktop\Projects\mi-app-PTS"
Add-MpPreference -ExclusionPath "C:\Users\Sylft\.ollama"
Add-MpPreference -ExclusionPath "C:\Users\Sylft\.cargo"
```

Puerto 11434 (Ollama)

Ollama corre en `localhost:11434`. Verificar que ningún otro proceso usa ese puerto. No requiere reglas de firewall externas — solo comunicación local entre Tauri y Ollama.

Micrófono en segundo plano (Porcupine)

Cuando la escucha continua está activa, Windows mostrará el ícono de micrófono en la barra de tareas permanentemente — mismo comportamiento que el `wakeword.js` actual en el navegador. Es el comportamiento esperado y correcto.

Primer build de Tauri — Tiempo estimado

OPERACIÓN	PRIMERA VEZ	SIGUIENTES
<code>cargo build</code> (debug)	10–20 min (descarga + compila crates)	~30 s (incremental)
<code>cargo build --release</code>	20–40 min	~2–5 min
<code>npm run tauri dev</code>	15–25 min	~1 min

Checklist de Instalación por Fase

FASE 0-1 (SETUP + CORE DESKTOP)

- ☐ Rust (rustup.rs)
- ☐ Node.js LTS
- ☐ Microsoft C++ Build Tools
- ☐ Tauri CLI (`npm install -g @tauri-apps/cli@^2`)
- ☐ VS Code + extensiones (rust-analyzer, Tauri)
- ☐ Drivers NVIDIA (verificar soporte RTX 5060 en Windows 10)

FASE 2 (CONTROL DEL SISTEMA)

- ☐ nvml-wrapper en Cargo.toml
- ☐ sysinfo en Cargo.toml

- ☐ windows crate en Cargo.toml

FASE 3 (VOZ LOCAL)

- ☐ CUDA Toolkit 12.x
- ☐ LLVM/Clang (`winget install LLVM.LLVM`)
- ☐ Python 3.11
- ☐ pip: faster-whisper · pyaudio · pvporcupine · kokoro-onnx · onnxruntime-gpu · edge-tts
- ☐ Modelo Whisper ggml-large-v3-turbo.bin descargado
- ☐ Cuenta Picovoice + AccessKey + modelo "Dalia" .ppn
- ☐ tauri://localhost agregado a orígenes Google Cloud

FASE 4 (IA LOCAL)

- ☐ Ollama instalado y verificado con GPU
- ☐ `ollama pull phi3.5` (primero, para validar)
- ☐ `ollama pull qwen2.5:7b`

FASE 6 (RAG + CLONACIÓN DE VOZ)

- ☐ pip: lancedb · sentence-transformers
- ☐ `ollama pull nomic-embed-text`
- ☐ pip: TTS (Coqui, para XTTS v2)
- ☐ cuDNN instalado y configurado